

Here's an updated version of the auction code that includes `std::promise` and `std::future`:

```
#include <iostream>
#include <thread>
#include <mutex>
#include <condition_variable>
#include <map>
#include <string>
#include <chrono>
#include <vector>
#include <atomic>
#include <future>

class Auction {
public:
    Auction(std::string item, int duration_seconds)
        : item_name(std::move(item)), duration(duration_seconds), highest_bid(0),
        auction_ended(false) {}

    // Function to place a bid on the item
    std::future<std::string> placeBid(int bid, const std::string& bidder) {
        std::lock_guard<std::mutex> lock(bid_mutex);

        std::promise<std::string> bid_promise;
        std::future<std::string> bid_future = bid_promise.get_future();

        if (auction_ended) {
            bid_promise.set_value("Auction for " + item_name + " has ended. No more bids allowed.");
            return bid_future;
        }

        if (bid > highest_bid) {
            highest_bid = bid;
            highest_bidder = bidder;
            std::string confirmation = "New highest bid for " + item_name + ": " +
std::to_string(bid) + " by " + bidder;
            bid_promise.set_value(confirmation); // Set confirmation message in promise
            std::cout << confirmation << std::endl;
        } else {
            std::string message = "Bid of " + std::to_string(bid) + " by " + bidder + " was too low
for " + item_name;
```

```

        bid_promise.set_value(message); // Set failure message in promise
        std::cout << message << std::endl;
    }

    return bid_future; // Return the future to the caller
}

// Function to run the auction with a time limit
void startAuction() {
    std::cout << "Starting auction for " << item_name << " lasting " << duration << "
seconds.\n";
    auto start_time = std::chrono::steady_clock::now();

    // Auction loop
    while (std::chrono::steady_clock::now() - start_time < std::chrono::seconds(duration)) {
        std::this_thread::sleep_for(std::chrono::milliseconds(100)); // Simulate time passing
    }

    // End the auction
    {
        std::lock_guard<std::mutex> lock(bid_mutex);
        auction_ended = true;
    }
    cv.notify_all(); // Notify all waiting threads that the auction has ended
    std::cout << "Auction for " << item_name << " has ended.\n";
}

// Function to get the final result of the auction
void waitForAuctionEnd() {
    std::unique_lock<std::mutex> lock(bid_mutex);
    cv.wait(lock, [this] { return auction_ended; });

    if (highest_bid > 0) {
        std::cout << "Auction result for " << item_name << ": Sold to " << highest_bidder
        << " for " << highest_bid << std::endl;
    } else {
        std::cout << "Auction result for " << item_name << ": No bids received.\n";
    }
}

private:
    std::string item_name;
    int duration;
    int highest_bid;

```

```

    std::string highest_bidder;
    bool auction_ended;
    std::mutex bid_mutex;
    std::condition_variable cv;
};

// Simulate bidding by different users
void simulateBidding(Auction& auction, const std::string& bidder, int max_bid) {
    for (int bid = 10; bid <= max_bid; bid += 10) {
        std::future<std::string> result = auction.placeBid(bid, bidder);

        // Asynchronously get the bid confirmation
        std::cout << "Confirmation for " << bidder << ": " << result.get() << std::endl;

        std::this_thread::sleep_for(std::chrono::milliseconds(150)); // Simulate time between bids
    }
}

int main() {
    Auction auction1("Painting", 5); // 5-second auction
    Auction auction2("Antique Vase", 7); // 7-second auction

    // Start auctions in separate threads
    std::thread auction_thread1(&Auction::startAuction, &auction1);
    std::thread auction_thread2(&Auction::startAuction, &auction2);

    // Simulate bidding in separate threads for each auction
    std::thread bidder1(simulateBidding, std::ref(auction1), "Alice", 100);
    std::thread bidder2(simulateBidding, std::ref(auction1), "Bob", 80);
    std::thread bidder3(simulateBidding, std::ref(auction2), "Charlie", 90);
    std::thread bidder4(simulateBidding, std::ref(auction2), "Dave", 120);

    // Wait for auctions to end and display results
    std::thread result_thread1(&Auction::waitForAuctionEnd, &auction1);
    std::thread result_thread2(&Auction::waitForAuctionEnd, &auction2);

    // Join all threads
    auction_thread1.join();
    auction_thread2.join();
    bidder1.join();
    bidder2.join();
    bidder3.join();
    bidder4.join();
    result_thread1.join();
}

```

```
    result_thread2.join();

    return 0;
}
```

Explanation of Updates

1. Using `std::promise` and `std::future` in `placeBid`:

- The `placeBid` method now creates a `std::promise` for each bid attempt. This promise holds a confirmation message (or a rejection message) for each bid.
- The future associated with the promise (`bid_future`) is returned to the caller, allowing asynchronous retrieval of the bid result.

2. Setting the Promise Value:

- If the bid is higher than the current highest bid, the promise's value is set to a confirmation message, which can be retrieved asynchronously.
- If the bid is too low or if the auction has ended, the promise's value is set to a failure message, allowing the caller to receive immediate feedback on the bid's status.

3. Asynchronous Bid Confirmation:

- In the `simulateBidding` function, each call to `placeBid` returns a `std::future`. By calling `result.get()`, we asynchronously retrieve the bid confirmation message, which may indicate success or failure based on the bid value and auction state.
- Using `result.get()` in this way provides a non-blocking way for each bidder to be notified of their bid status.

4. Thread Safety and Synchronization:

- `std::mutex` and `std::lock_guard` ensure thread-safe access to the highest bid and highest bidder data.
- `std::condition_variable` is used to signal when the auction ends, allowing the main thread to display final results.

Summary

By incorporating `std::promise` and `std::future`, this version of the auction system allows for asynchronous bid updates and confirmations, enhancing the responsiveness and interactivity of the system. Each bid can be confirmed in real-time, providing immediate feedback to bidders, while ensuring that the highest bid data remains protected with `std::mutex` and `std::lock_guard`. The use of `std::condition_variable` continues to handle auction end notifications, keeping the system synchronized and efficient.